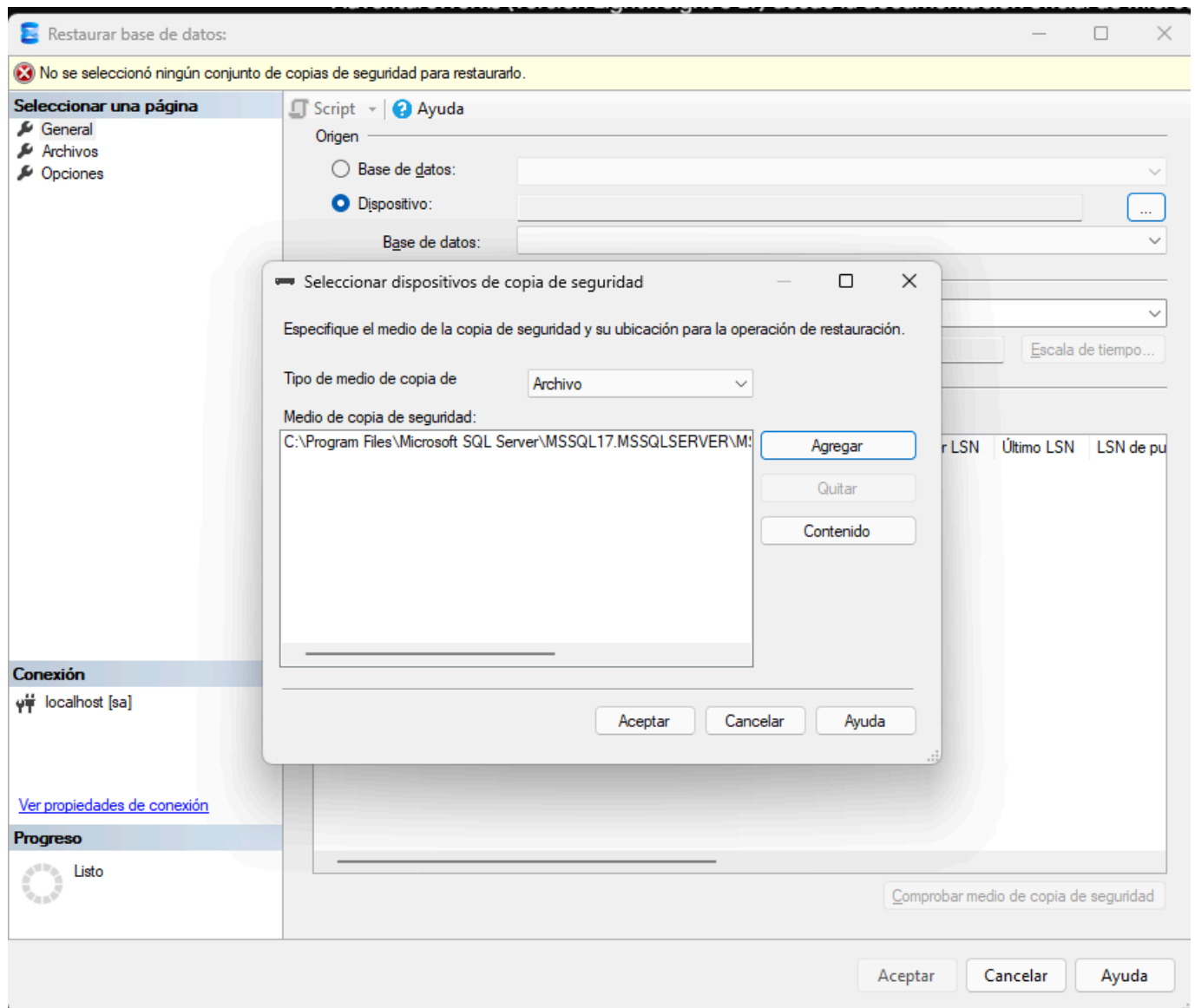


Implementación de objetos de programabilidad con SQL

Requisitos previos

Primero descargamos la bbdd que necesitamos para el lab.



Conexión con AdventureWorksLT

la restaure y la puse en esta direccion q nos indica la web oficial de microsoft 'C:\Program Files\Microsoft SQL Server\MSSQL17.MSSQLSERVER\MSSQL\Backup'

SQLQuery1.sql...25 (sa (78))*

```
1  -- Verify key tables in AdventureWorksLT
2  SELECT TOP (5) CustomerID, FirstName, LastName
3  FROM SalesLT.Customer;
4
5  SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
6  FROM SalesLT.SalesOrderHeader;
7
8  SELECT TOP (5) ProductID, Name, ListPrice
9  FROM SalesLT.Product;
```

100 % No se encontraron problemas.

Resultados Mensajes

	CustomerID	FirstName	LastName
1	1	Orlando	Gee
2	2	Keith	Harris
3	3	Donna	Carreras
4	4	Janet	Gates
5	5	Lucy	Harrington

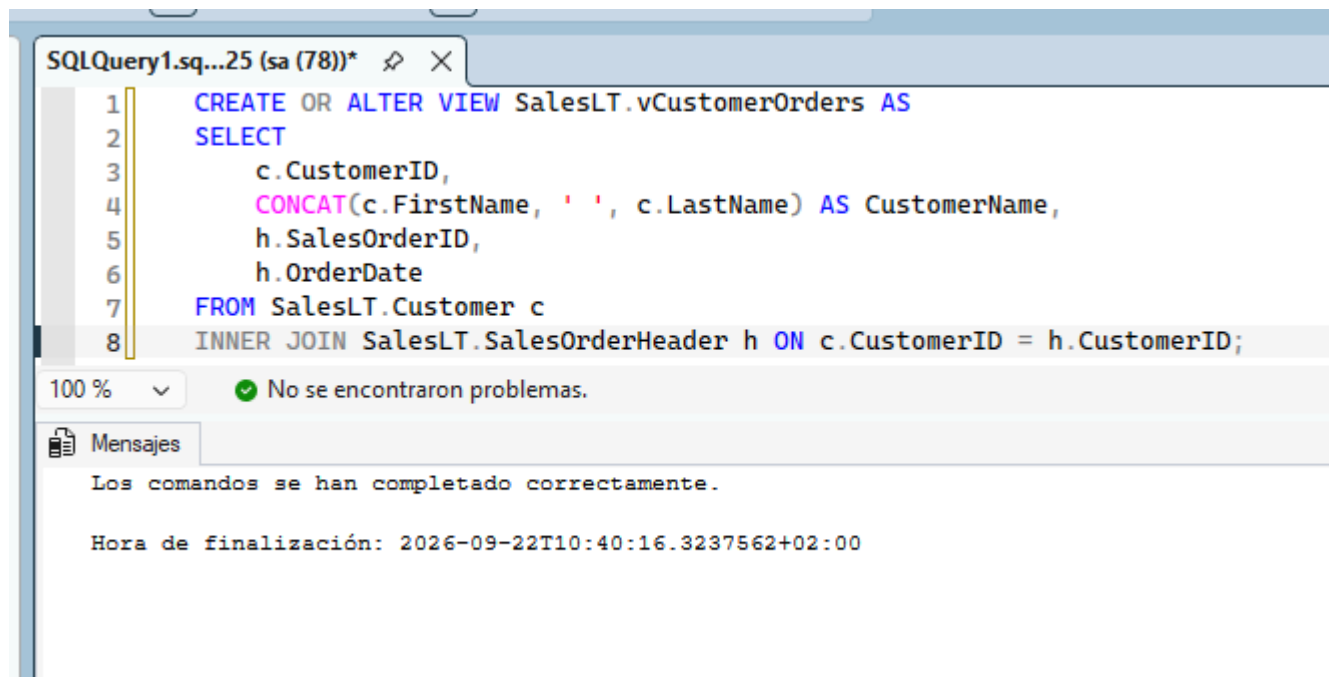
	SalesOrderID	OrderDate	CustomerID
1	71774	2008-06-01 00:00:00.000	29847
2	71776	2008-06-01 00:00:00.000	30072
3	71780	2008-06-01 00:00:00.000	30113
4	71782	2008-06-01 00:00:00.000	29485
5	71783	2008-06-01 00:00:00.000	29957

	ProductID	Name	ListPrice
1	690	UL Road Frame, Black, 58	1421.50

Consulta ejecutada correctamente.

Creación de una vista para simplificar las consultas

Creé una vista que combina los clientes con sus pedidos en el esquema **SalesLT**. De esta forma, dejé preparada una consulta reutilizable y oculté la complejidad de las relaciones entre tablas.



The screenshot shows a SQL query window titled "SQLQuery1.sql...25 (sa (78))*". The query is as follows:

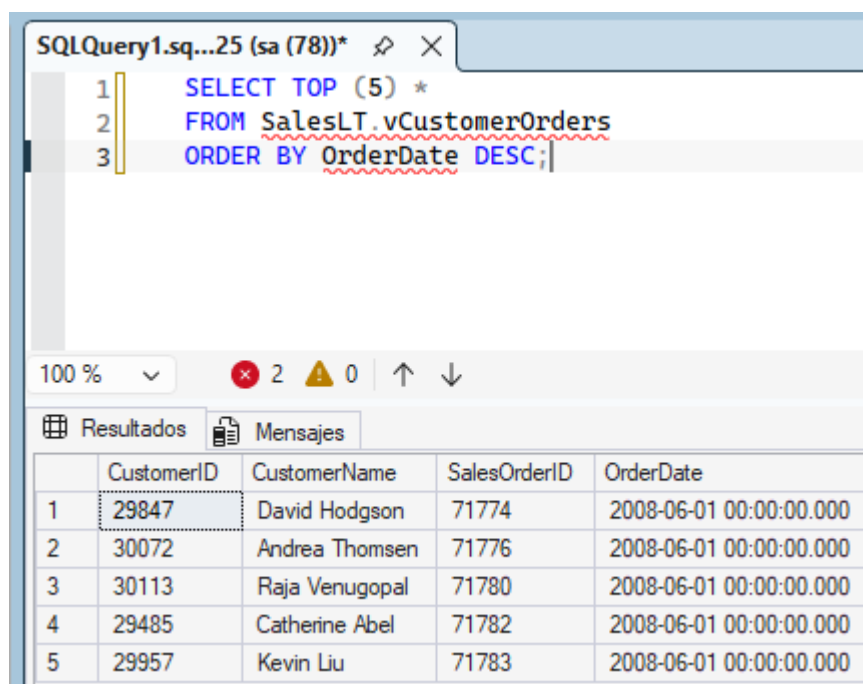
```
1 CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
2 SELECT
3     c.CustomerID,
4     CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
5     h.SalesOrderID,
6     h.OrderDate
7 FROM SalesLT.Customer c
8 INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
```

Below the query, the status bar indicates "100 %" and "No se encontraron problemas." (No problems were found). The "Mensajes" (Messages) pane shows the following text:

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:40:16.3237562+02:00

A continuación, validé que la vista se hubiera creado correctamente y que devolviera los datos esperados.



The screenshot shows a SQL query window titled "SQLQuery1.sql...25 (sa (78))*". The query is as follows:

```
1 SELECT TOP (5) *
2 FROM SalesLT.vCustomerOrders
3 ORDER BY OrderDate DESC;
```

Below the query, the status bar indicates "100 %", "2" errors, and "0" warnings. The "Resultados" (Results) pane shows the following data:

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

Creación de un procedimiento almacenado para procesar un pedido

Encapsulé en un procedimiento almacenado la operación de añadir una línea a un pedido existente en AdventureWorksLT. El procedimiento también actualiza el subtotal de la cabecera del pedido.

```
SQLQuery1.sql...25 (sa (78))*
1 CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem
2     @SalesOrderID INT,
3     @ProductID INT,
4     @Quantity INT
5 AS
6 BEGIN
7     SET NOCOUNT ON;
8     BEGIN TRANSACTION;
9
10    -- Use Product ListPrice as UnitPrice
11    DECLARE @UnitPrice DECIMAL(18,2);
12    SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
13    FROM SalesLT.Product
14    WHERE ProductID = @ProductID;
```

100 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:41:17.8787401+02:00

Después ejecuté el código T-SQL de prueba para comprobar que el procedimiento funcionaba correctamente.

```
SQLQuery1.sql...25 (sa (78))*
3 FROM SalesLT.SalesOrderHeader
4 ORDER BY SalesOrderID DESC);
5 EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,
6     @ProductID = 680,
7     @Quantity = 1; -- adjust ProductID as needed
8
9 SELECT TOP (5) *
10 FROM SalesLT.SalesOrderDetail
11 WHERE SalesOrderID = @SalesOrderID
12 ORDER BY SalesOrderDetailID DESC;
13
14 SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
15 FROM SalesLT.SalesOrderHeader
16 WHERE SalesOrderID = @SalesOrderID;
```

100 % 1 0

Resultados Mensajes

	SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	LineTotal	rowguid
1	71946	113407	1	680	1431,50	0,00	1431,500000	5CED70DB-CF78-4D40-B821-218F480E7D27
2	71946	113406	1	916	31,584	0,00	31,584000	B571CEE0-A90A-4E76-9796-379B55352A28

	SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
1	71946	1463,084	3,1163	0,9738	1467,1741

Creación de una función escalar para realizar cálculos reutilizables

Creé una función escalar que calcula y devuelve el valor total de un pedido a partir de los importes de sus líneas en AdventureWorksLT.

```
SQLQuery1.sql...25 (sa (78))*
1      CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
2      RETURNS DECIMAL(18,2)
3      AS
4      BEGIN
5          DECLARE @Total DECIMAL(18,2);
6
7          SELECT @Total = SUM(LineTotal)
8          FROM SalesLT.SalesOrderDetail
9          WHERE SalesOrderID = @OrderID;
10
11         RETURN ISNULL(@Total, 0.00);
12     END;
```

100 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:42:24.4966879+02:00

Finalmente, ejecuté el código T-SQL necesario para utilizar la función y comprobar el total calculado.

```
SQLQuery1.sql...25 (sa (78))*
1      SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
2      FROM SalesLT.SalesOrderDetail d
3      GROUP BY d.SalesOrderID
4      ORDER BY d.SalesOrderID DESC;
```

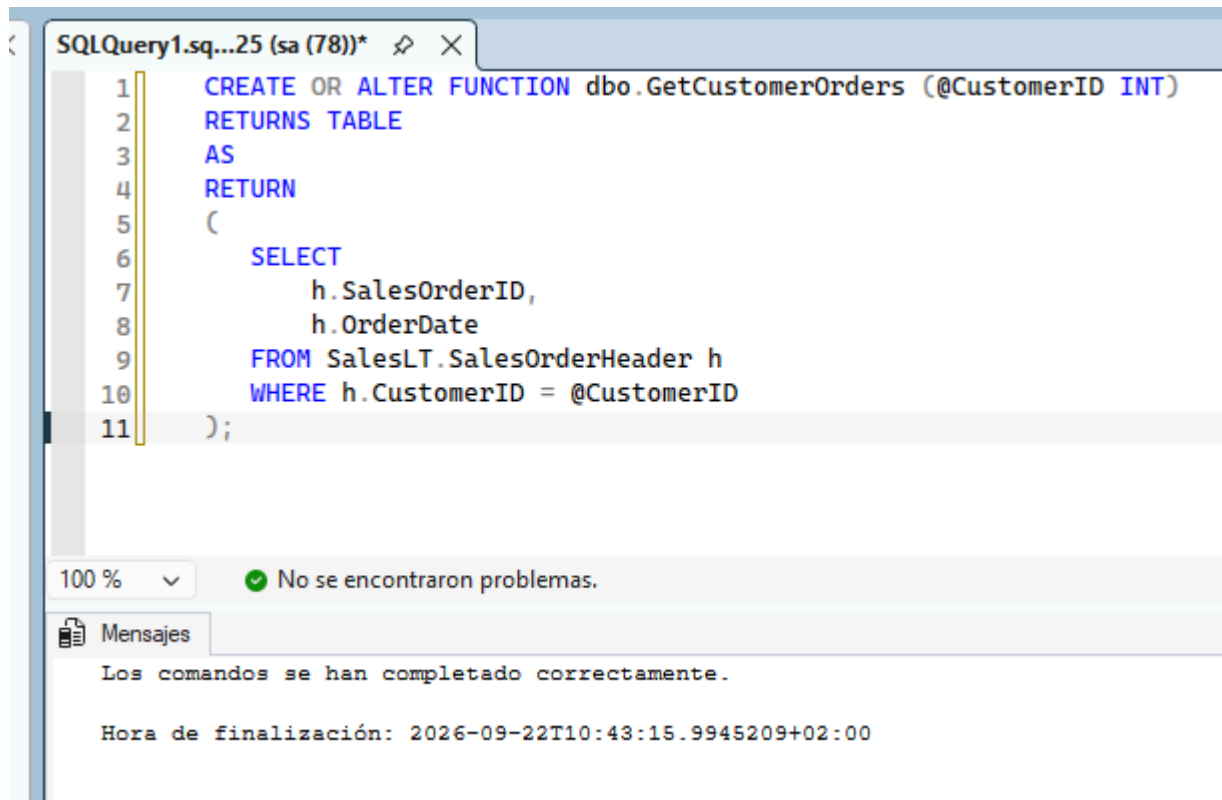
100 % 1 0

Resultados Mensajes

	SalesOrderID	OrderTotal
1	71946	1463.08
2	71938	74160.23
3	71936	79589.62
4	71935	5533.87
5	71923	96.11
6	71920	2527.13

Creación de una función con valores de tabla en línea (TVF)

Construí una función con valores de tabla que devuelve los pedidos de un cliente concreto de AdventureWorksLT. Este tipo de función se puede utilizar directamente en consultas **SELECT** y en combinaciones **JOIN**.



The screenshot shows a SQL query window titled "SQLQuery1.sql...25 (sa (78))*". The query is a T-SQL statement to create or alter a function named "dbo.GetCustomerOrders" that takes an integer parameter "@CustomerID". The function returns a table and contains a SELECT statement that retrieves "SalesOrderID" and "OrderDate" from the "SalesLT.SalesOrderHeader" table for a specific customer. The status bar indicates "100 %", "No se encontraron problemas.", and a message pane confirms successful execution.

```
1 CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
2 RETURNS TABLE
3 AS
4 RETURN
5 (
6     SELECT
7         h.SalesOrderID,
8         h.OrderDate
9     FROM SalesLT.SalesOrderHeader h
10    WHERE h.CustomerID = @CustomerID
11 );
```

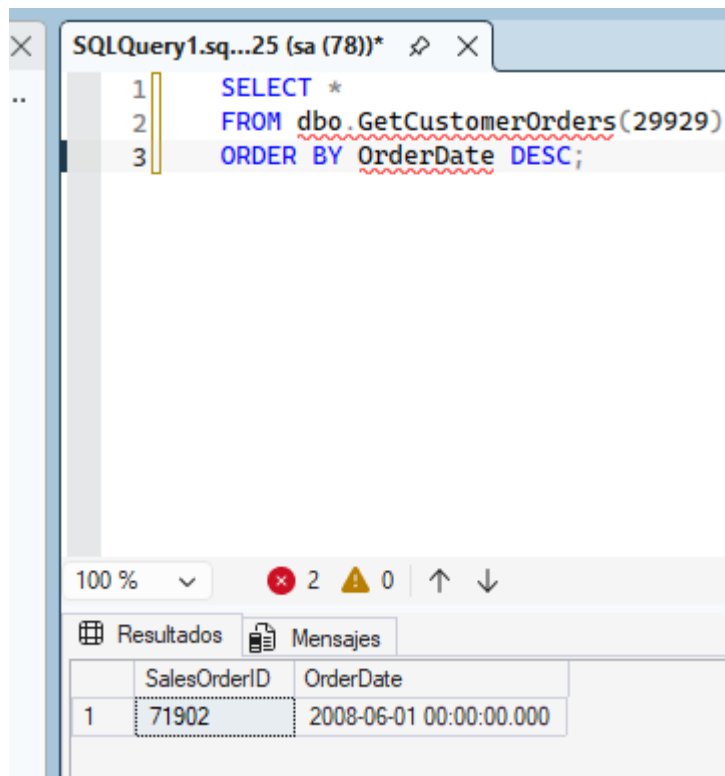
100 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:43:15.9945209+02:00

Ejecuté el código T-SQL para consultar la función y revisar los pedidos devueltos.



The screenshot shows a SQL query window titled "SQLQuery1.sql...25 (sa (78))*". The query is a T-SQL statement to select all columns from the "dbo.GetCustomerOrders" function for customer ID 29929, ordered by "OrderDate" in descending order. The status bar indicates "100 %", "2" errors, and "0" warnings. The "Resultados" pane shows a single row with "SalesOrderID" 71902 and "OrderDate" 2008-06-01 00:00:00.000.

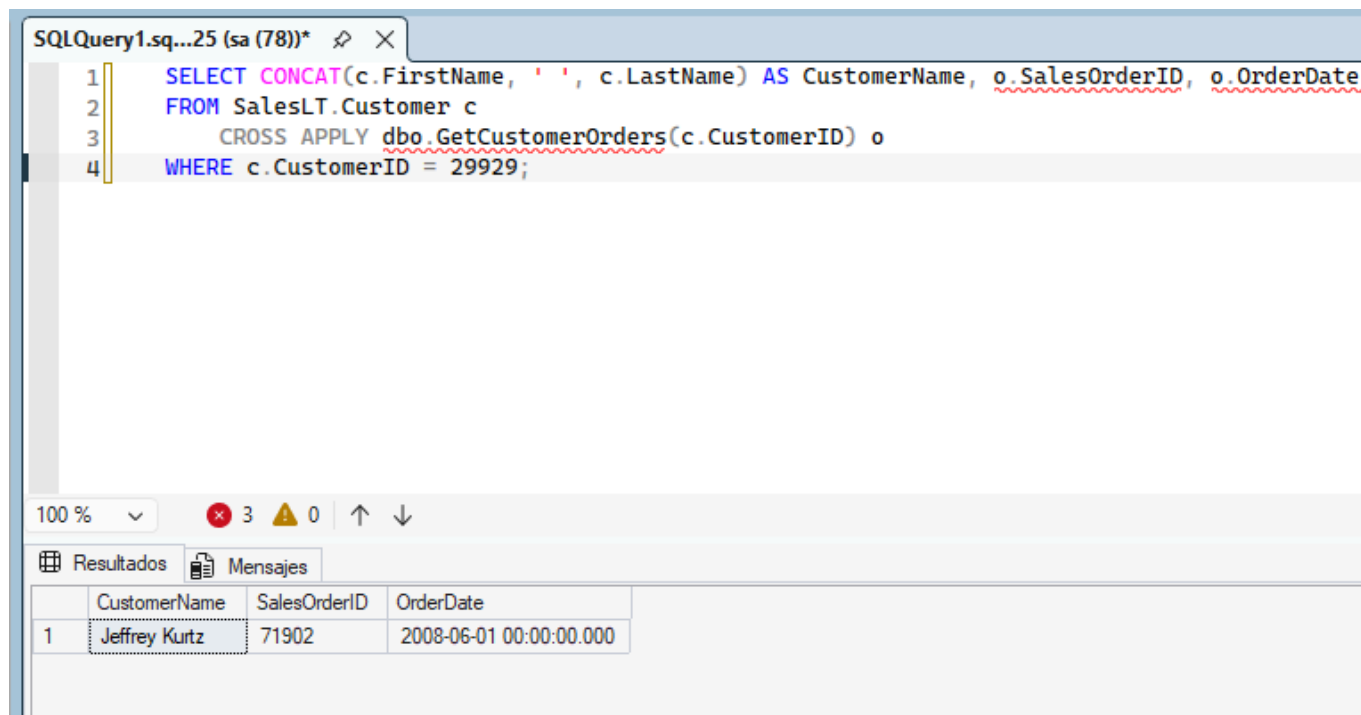
```
1 SELECT *
2 FROM dbo.GetCustomerOrders(29929)
3 ORDER BY OrderDate DESC;
```

100 % 2 0

Resultados Mensajes

	SalesOrderID	OrderDate
1	71902	2008-06-01 00:00:00.000

Después combiné la función con la tabla de clientes para obtener la información relacionada.



The screenshot shows a SQL query window titled "SQLQuery1.sql...25 (sa (78))*". The query is as follows:

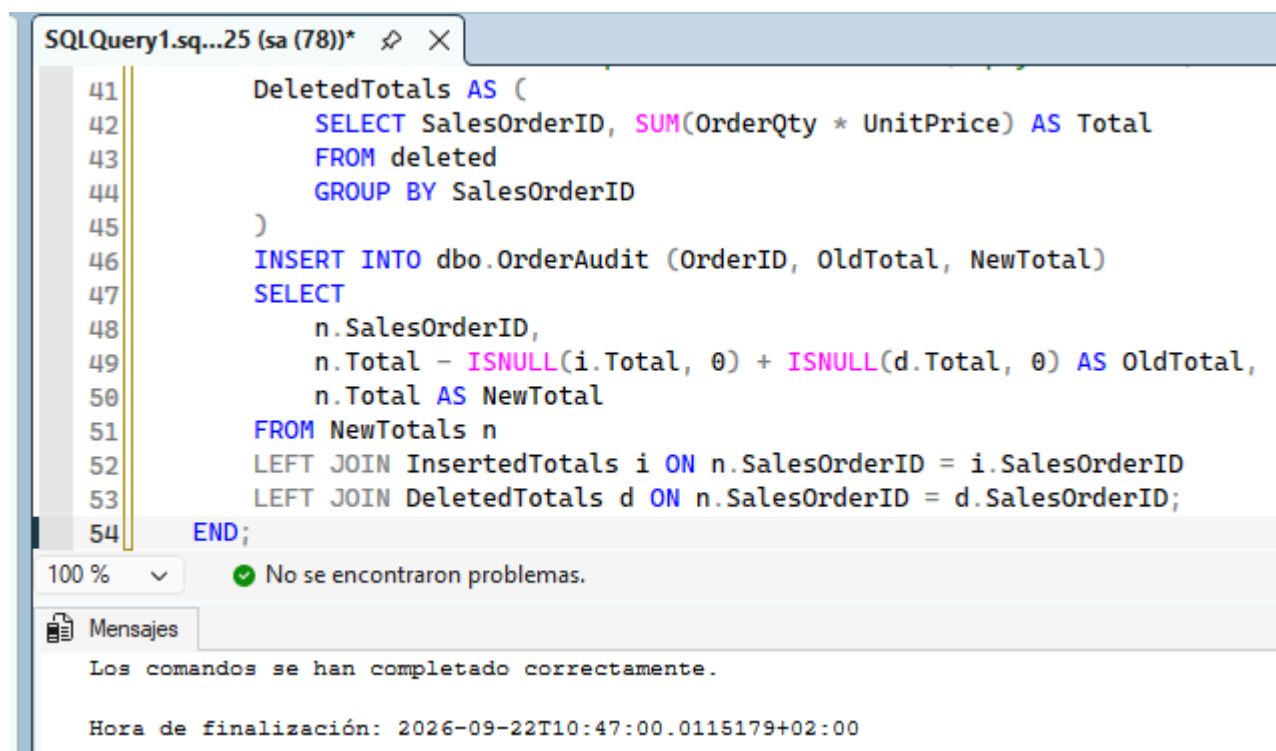
```
1 SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID, o.OrderDate
2 FROM SalesLT.Customer c
3 CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
4 WHERE c.CustomerID = 29929;
```

Below the query window, the "Resultados" (Results) tab is active, displaying a table with the following data:

	CustomerName	SalesOrderID	OrderDate
1	Jeffrey Kurtz	71902	2008-06-01 00:00:00.000

Creación de un trigger para registrar cambios

Añadí un trigger para registrar las actualizaciones de los totales de los pedidos cuando se modifican los detalles de los pedidos del esquema `SalesLT`. Así, los cambios quedan controlados automáticamente.



The screenshot shows a SQL query window titled "SQLQuery1.sql...25 (sa (78))*". The query is as follows:

```
41 DeletedTotals AS (
42     SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
43     FROM deleted
44     GROUP BY SalesOrderID
45 )
46 INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)
47 SELECT
48     n.SalesOrderID,
49     n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
50     n.Total AS NewTotal
51 FROM NewTotals n
52 LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
53 LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
54 END;
```

Below the query window, the "Mensajes" (Messages) tab is active, displaying the following messages:

100 % No se encontraron problemas.

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:47:00.0115179+02:00

Por último, probé el trigger realizando una modificación y comprobé que se registraba correctamente.

SQLQuery1.sql...25 (sa (78))*

2

3

4

5

6

7

8

9

UPDATE d

SET OrderQty = OrderQty + 1

FROM SalesLT.SalesOrderDetail d

WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader ORDER BY SalesOrderID DESC);

SELECT TOP (5) *

FROM dbo.OrderAudit

ORDER BY AuditID DESC;

100 %

2

0

↑

↓

Resultados

Mensajes

	AuditID	OrderID	OldTotal	NewTotal	ChangedAt
1	1	71946	1463.08	2926.17	2026-09-22 08:47:17.1179341